Intro Generative AI

Boolean Master · Lezione 10

Tre settimane di attrezzi

Tutto quello che avete costruito finora serviva a questo.

SETT. 1

Python base

Setup, collezioni, funzioni, moduli, pathlib.

SETT. 2

OOP e dati

Classi, dataclasses, eccezioni, parsing JSON.

SETT. 3

Pydantic e async

Validazione di schemi, async e httpx per le API.

SETT. 4 · OGGI

Generative AI

La macchina dall'altra parte delle API.

INSIGHTS **Pydantic** e **async** non erano esercizi: sono gli attrezzi per le API AI · sabato la prima chiamata a un LLM da Python

Oggi apriamo il cofano

Niente codice oggi: costruiamo il modello mentale per tutto il resto del corso.

01

Da dove arrivano

Una storia breve, con una data precisa.

02

Cosa fanno davvero

Il prossimo token, una parola alla volta.

03 · DEMO LIVE

I token

Di cosa sono fatti i testi, dal vivo.

04

Dentro il Transformer

Embeddings e attention, senza matematica.

05

La context window

Il limite fisico del modello.

1. Da dove arrivano gli LLM

Una storia breve, con una data precisa



Prima degli LLM

Decenni di NLP, un sistema diverso per ogni compito.

Regole scritte a mano

Grammatiche e dizionari compilati da esperti, fragili fuori dai casi previsti.

Modelli statistici

Conteggi e probabilità su corpora, meglio ma sempre limitati.

Un sistema per ogni compito

Uno per tradurre, uno per classificare, uno per estrarre nomi.



Cosa è cambiato?

2017: Attention Is All You Need

La data che conta: nasce l'architettura Transformer.

PAPER · 2017

Attention Is All You Need

Vaswani et al.

Introduce l'architettura **Transformer**, la base di tutto quello che usiamo oggi.

BERT GPT-2 GPT-3 ChatGPT ...

INSIGHTS il **Transformer** (2017) è il fondamento di tutti i modelli attuali · da quel momento la corsa è una corsa di **scala**

La scala sale in fretta

In tre anni, cento volte più grande. E con la scala emergono capacità nuove.

GPT-2 `2019 · 1,5B parametri`



Testo così convincente che OpenAI ne *ritarda il rilascio completo* per timore di abusi.

GPT-3 `2020 · 175B parametri`

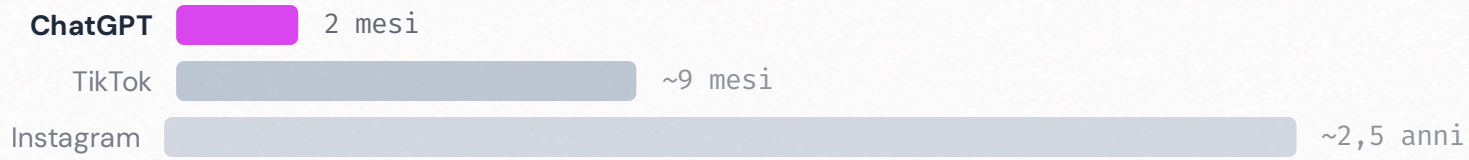


Scrive saggi e codice *senza addestramento specifico per compito*: capacità emergenti.

Novembre 2022: il momento ChatGPT

Non un modello più grande: un modello addestrato a dialogare.

100M utenti in 2 mesi



GPT-3.5 + interfaccia chat + addestramento a dialogare (*RLHF*, ci torniamo) · poi **GPT-4** (2023) e l'ondata attuale: **multimodal** e **reasoning models**

INSIGHTS ChatGPT non era un modello più grande: era addestrato a **dialogare** · l'app consumer a crescita più rapida della storia (UBS)

2. Cosa fa davvero un LLM

Una macchina di completamento



Una macchina di completamento

Gli dai un inizio, continua nel modo più probabile.

Il gatto è salito sul ___

tetto · 81% tavolo · 12% treno · 2%

Tutta la risposta è una *sequenza di predizioni*, una parola alla volta. Significa che "capisce" nel senso umano?

INSIGHTS un LLM **completa**: tutta la risposta è una sequenza di predizioni · tutto quello che vedremo oggi discende da questo principio



La risposta nasce una parola alla volta

A ogni passo il modello rilegge tutto e sceglie solo la parola successiva.

`Qual è la capitale della Francia? → ___`	**La**	P = 0,92
`... → La ___`	**capitale**	P = 0,88
`... → La capitale ___`	**della**	P = 0,95
`... → La capitale della ___`	**Francia**	P = 0,97
`... → La capitale della Francia ___`	**è**	P = 0,93
`... → La capitale della Francia è ___`	**Parigi**	P = 0,99
La capitale della Francia è Parigi	risposta completa	

La risposta non è mai esistita come piano: è **emersa** un pezzo alla volta.

Predire è classificare

A ogni passo la scelta è un problema di **classificazione** su un vocabolario di circa **100.000 token**, ognuno con la sua probabilità.

```
La capitale della Francia è ___
```

E la prima conseguenza si vede da sola: la scelta ha una *componente probabilistica*, per questo la **stessa domanda può dare risposte diverse**.

Token	Probabilità
Parigi	**92,1%**
Roma	2,4%
la	1,8%
situata	0,7%
Lione	0,3%
…	…
~100.000 altri	*< 0,1%*

Le allucinazioni non sono un guasto

Sono il funzionamento normale, visto dal lato sbagliato.

`il completamento più probabile`

… è anche vero

Sembra che il modello "sappia". La risposta è corretta e sicura.

… è solo plausibile

Il modello **inventa**, con la stessa identica sicurezza.

Il modello non "sa" e poi parla: **completa**. Stesso meccanismo, due esiti.

`INSIGHTS` il modello non sa e poi parla: **completa** · vero e plausibile escono con la stessa sicurezza, per questo si **verifica sempre**

Da completatore ad assistente

Se il modello completa, perché ChatGPT risponde alle domande?

FASE 1

Pre-training

Impara a **completare**: quantità enormi di testo, un solo obiettivo, predire il prossimo token.

→

FASE 2

RLHF

Impara a **dialogare**: da completatore ad assistente che risponde.

Il cuore resta lo stesso: predire il prossimo token.

3. I token

Di cosa sono fatti i testi che il modello legge e scrive

Non parole: token

Il modello non ragiona in parole ma in token, pezzi di parole.

`"ciao"`	`ciao`	1 token: parola comune
`"intelligenza"`	`intell` `igenza`	2 token: si spezza
`"ChatGPT"`	`Chat` `G` `PT`	3 token: parola rara

L'unità di lavoro del modello è il **token**, non la parola. Meglio vederlo dal vivo.



Demo

DEMO DAL VIVO **`platform.openai.com/tokenizer`**

1. Una frase semplice **in inglese**: quasi ogni parola un token
2. La stessa frase **in italiano**: più token a parità di contenuto
3. Una parola lunga italiana: `fatturazione`
4. Uno **snippet Python** di due righe: il codice si tokenizza bene
5. Una **parola inventata**

Bonus per l'aula: perché i modelli storicamente sbagliavano a contare le R di `strawberry` ?



Cosa ci dice la demo

Come lo vediamo noi:

s t r a w b e r r y

Lettere. Le tre R si contano a colpo d'occhio.

Come lo vede il modello:

`str` `aw` `berry`

Token. Le lettere non esistono.

L'**inglese** è la lingua "nativa" dei tokenizer: l'italiano si spezza di più, e quindi **costa di più**.

Per capire un comportamento strano del modello, chiediti **cosa vede davvero**.

Perché i token contano

Non sono un dettaglio implementativo.

COSTO

Le API si pagano a token

Input e output: ogni token ha un prezzo. Lingue che si spezzano di più costano di più.

FINESTRA

La context window si misura in token

Il limite di quello che il modello vede è un budget di token, non di pagine.

Deep dive su costi e ottimizzazione: **Modulo 2**, lezione dedicata a token e costi.

4. Dentro il Transformer

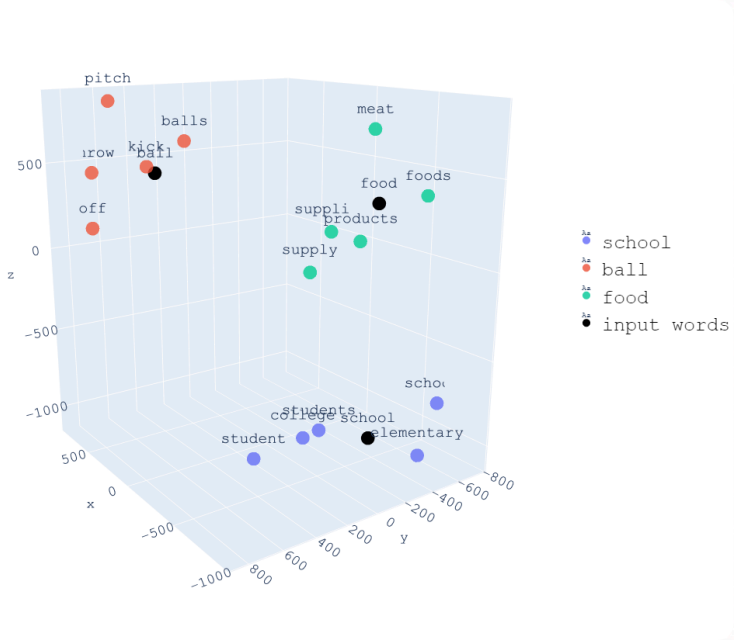
Alto livello, niente matematica, tre idee

Embeddings: il significato come coordinate

Ogni token viene trasformato in **coordinate** in uno spazio a molte dimensioni.

- Parole con significato simile finiscono **vicine**
- È così che il modello "capisce" *sinonimi e intenti*, senza regole scritte a mano

```
"endpoint" ↔ vicino a "API"
"endpoint" ↮ lontano da "fattura"
```



Esplorazione interattiva: projector.tensorflow.org · Deep dive nel Modulo 2 (ricerca semantica)

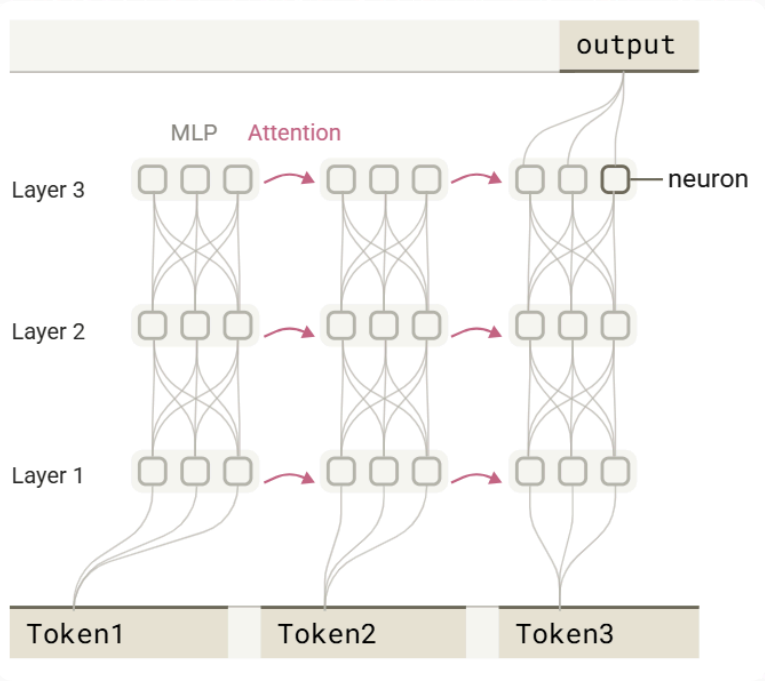
Attention: guardare indietro

Il meccanismo che dà il nome al paper del 2017.

Il modello elabora il testo **token per token**, attraverso una pila di layer:

- A ogni layer, ogni token può "guardare" i token **che lo precedono**
- Si crea un flusso di informazione dal passato al presente

Il diagramma mostra come ogni token, a ogni layer, ha accesso all'elaborazione dei token precedenti.

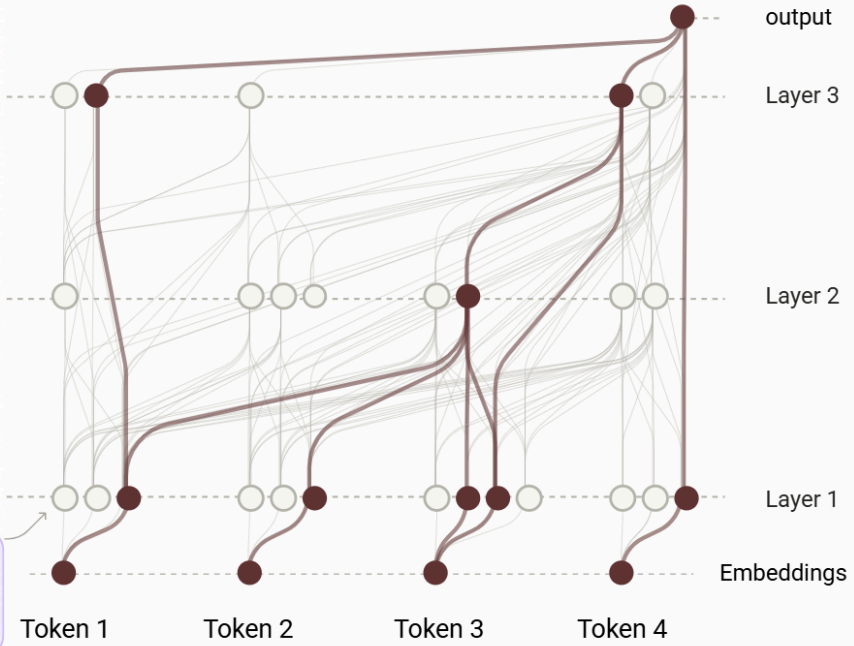


Attention: scegliere cosa pesa

Non tutte le parole del contesto valgono uguale: il modello impara a dare **più peso alle parti rilevanti**.

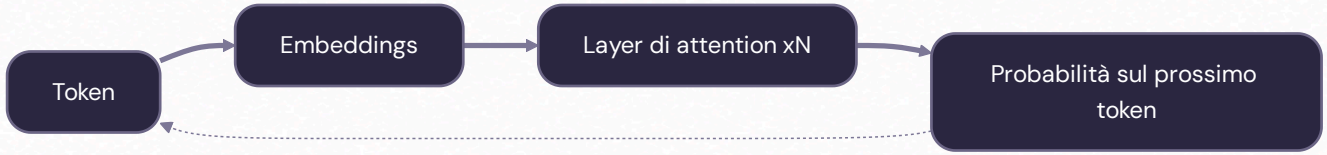
- Quando genera "Parigi" si concentra su "capitale" e "Francia", anche se **lontane nel testo**
- È questo che permette *coerenza* su contesti lunghi migliaia di token

L'attention pesa le parti rilevanti del contesto · è ciò che tiene coerente un testo lungo



Il Transformer in un diagramma

Questo loop è tutto quello che succede quando il modello "scrive".



La freccia di ritorno: il token scelto rientra nel contesto e il giro riparte. Adesso sapete leggere il Transformer ad alto livello, e per il resto del corso basta questo.

INSIGHTS embeddings, attention, probabilità sul prossimo token: il **loop completo** · questo livello di lettura basta per tutto il corso

L’embedding: la parola diventa numeri

Un computer non legge le parole, legge i **numeri**: l'embedding trasforma ogni parola in numeri che ne catturano il *significato*.

"porta" →	0,2	0,8	0,3	1,1	…
"tavolo" →	0,1	0,9	0,4	1,0	…
"gatto" →	0,9	-0,4	1,2	0,1	…

`porta` e `tavolo` → numeri simili · `gatto` → numeri diversi



Uno spazio **2D** (semplificato): i significati simili si **raggruppano**.

La probabilità: quale parola viene adesso?

All'uscita il modello dà un **voto** a ogni parola: quanto è probabile che sia la prossima.



Una probabilità per ognuna delle ~100.000 parole; insieme fanno **1**. Vince la più alta → `porta` .

Due vettori, due momenti

Stessa parola, due numeri diversi: uno **entra** nel modello, uno **esce**.

ENTRA · IL SIGNIFICATO

"porta"

↓

[0,2 · 0,8 · 0,3 · 1,1 · …]

che cos'è «porta»

ESCE · LA SCELTA

"chiudere la ___"

↓

[… · **porta 0,78** · …]

quanto «porta» è probabile adesso

L'embedding è **come il modello legge**. La probabilità è **come sceglie cosa scrivere**.

5. La context window

Il limite fisico del modello

Il modello non ricorda

La "memoria" delle chat è un'illusione costruita bene.

USER Come valido questo schema?

ASSISTANT Puoi usare un validator Pydantic...

USER E per i campi opzionali?

↑ a ogni nuovo messaggio, il modello rilegge TUTTO questo, da capo

Nuova chat

Nessun messaggio.
Contesto perso.

L'anatomia dei messaggi (system, user, assistant) la vediamo **giovedì**: oggi basta l'idea.

INSIGHTS a ogni messaggio il modello **rilegge l'intera conversazione** da capo · nuova chat = reset: non esiste memoria, esiste il contesto



Stessa domanda, due contesti

"Che formato avevamo scelto per l'export?"

CONTESTO PULITO

User: Per il modulo di export il cliente vuole il **formato XML**.

Assistant: Ok, preparo lo schema XML.

User: La deadline è venerdì 15.

Assistant: Schema XML pronto per venerdì 15.

Risposta facile da trovare.

CONTESTO SOVRACCARICO

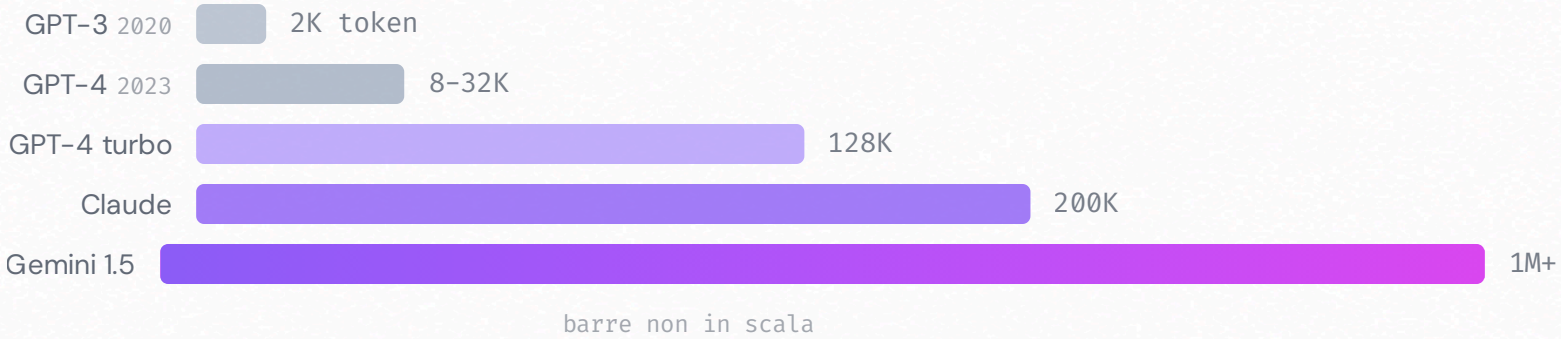
User: Allora, la situazione è questa: il team ha fatto una riunione la settimana scorsa e abbiamo discusso timeline, budget, risorse, requisiti funzionali e non funzionali, e la scelta degli strumenti. Marco preferiva PostgreSQL per il database, Giulia suggeriva MongoDB, alla fine abbiamo scelto PostgreSQL. Poi abbiamo parlato del frontend e abbiamo scelto React con TypeScript. Il designer ha mandato i mockup ma non sono ancora approvati. Il PM ha detto che dobbiamo considerare anche l'integrazione col sistema legacy che usa SOAP, circa 200 endpoint e documentazione parziale. L'infrastruttura sarà su AWS sotto i 3.000 euro al mese, CI/CD con GitHub Actions, ambienti di staging separati per il QA. Tornando al modulo di export, che è la parte che mi interessa adesso: il cliente vuole generare fatture in batch, un sistema di notifiche email e il **formato XML per l'export**, più template personalizzabili e compatibilità SDI. Performance: almeno 1.000 fatture al minuto. Audit logging su tutte le operazioni, GDPR compliance, retention 10 anni. C'è anche la migrazione di 2 milioni di fatture dal vecchio sistema in CSV proprietario, se ne occupa Sara. Poi parliamo di autenticazione: OAuth2 con Keycloak, realm nuovo, ruoli admin/contabile/viewer, SSO con Active Directory del cliente su Windows Server 2019, password policy 12 caratteri con rotazione a 90 giorni, timeout sessione 30 minuti, 2FA per gli admin…

Difficile: per noi e per il modello.

Più rumore c'è, più è difficile *pesare le parti giuste*: l'attention non fa miracoli.

Quanto è grande la finestra

Ma grande non vuol dire gratis.



INSIGHTS da 2K a oltre 1M di token in pochi anni · ma più contesto inutile = risposte peggiori: le cose importanti **si perdono nel mezzo**

Il contesto è una risorsa

Le prime tre regole operative del corso.

01 · GESTISCILO

Non è infinito né gratis

Sii consapevole di quanto contesto stai usando, e con cosa lo stai riempiendo.

02 · POSIZIONA

Inizio o fine

Le informazioni rilevanti all'inizio o alla fine del prompt: è lì che il modello le trova meglio.

03 · RESETTA

Nuova conversazione

Il modo più semplice per ripartire con un contesto pulito.

INSIGHTS il contesto è una **risorsa da gestire**, non un cestino · all'inizio o alla fine: è lì che il modello trova meglio le cose

Il modello mentale

Questa slide è la lezione: se tra un mese ricordate questa catena, ricordate tutto.

CAP. 2

Completa

Token dopo token, il più probabile.

→

CAP. 3

Token

Pezzi di parole, l'unità di tutto.

→

CAP. 4

Attention

Sceglie a cosa dare peso.

→

CAP. 5

Context window

La risorsa da gestire.

Quiz

A voce, con l'aula. Le risposte le avete già viste tutte.

1. Perché lo stesso prompt può dare **risposte diverse**?
2. Cosa sono i **token**?
3. Perché una conversazione molto lunga **peggiora le risposte**?
4. A cosa serve l'**attention**?

Giovedì e sabato

Oggi avete visto che il modello completa il contesto che gli date. Quindi il modo in cui chiedete conta.

GIOVEDÌ · LEZIONE 11

Prompt Engineering

Il modo in cui chiedi conta: impariamo a scrivere il contesto che il modello completa.

SABATO · LEZIONE 12

Prima applicazione AI

La prima chiamata a un LLM via API da Python: il cerchio si chiude.

`Pydantic` `async` `httpx`

INSIGHTS giovedì: impariamo a **scrivere bene il contesto** · sabato: la prima chiamata API, dove gli attrezzi Python smettono di essere esercizi